Optimising New York Citi Bike Station Occupancy: A CTMC Model with Truck-Rebalancing-Rate Allocation

Spencer Arnold, Paris Phan, Garrett Uthlaut
University of Virginia, Charlottesville, Virginia

***Abstract*—New York City's Citi Bike system suffers a form of spatial imbalance during commuter peaks: source stations near residential blocks run empty as the public from home, and destination stations near business districts overfill, causing two failures that force riders to abandon the system. We model each station as a finite-capacity continuous-time Markov chain (an M/M/1/K queue), extended with a Poisson truck-reset process that returns the station to a target fill level. Per-station failure rates are computed in closed form when no rebalancing is applied, or by direct linear solution of the stationary equation when rebalancing is added. Using data of a five-station Midtown Manhattan cluster fit to a December 2021 weekday morning, we then solve the budget-constrained problem of allocating a total truck-visit-rate maximum across the cluster to minimise long-run user failures. At an operating budget of five truck-visits per hour across the cluster, the optimal allocation cuts long-run user failures from 41.87 to 1.77 events per hour, a 95.8 percent reduction; a Pareto-frontier sweep places the operational knee at 5.71 truck-visits per hour. We then report and discuss validation alongside the limitations of the model assumption.**

***Keywords*—bike-sharing, M/M/1/K queue, continuous-time Markov chain, rebalancing, marginal analysis, resource allocation**

I. Introduction

Citi Bike is the largest bike-sharing system in North America, operating roughly 25,000 bikes across more than 1,500 docking stations in the New York metropolitan area. However, data suggests that these station placements and capacity do not adequately account for areas of greater attraction. As a result, docks in dense, high-traffic areas are often empty while other docks are completely full. We refer to these two failure modes as *stockout* (no bikes available for withdrawal) and *dockblock* (no docks available for return). Both of these failures convert a successful trip into a walk, a wait or an abandoned ride. During morning rush, directional commute flows produce predictable imbalances: residential-area stations drain while business-district stations overfill. This project looks to model and quantify these dynamics using stochastic processes, with the goal of evaluating strategies to improve performance.

The standard operational lever is *truck rebalancing*: vehicles dispatched to redistribute bikes between stations. We pose the question: given a total budget of truck-visit *rates* across a station cluster, how should that resource be allocated across stations to minimize the long-run rate of user failures, and at what budget threshold does additional rebalancing stop paying for itself? The decision variable is the rate θ_n at which station n is rebalanced.

We model the research system as (i) a continuous-time Markov chain (CTMC) model for a single Citi Bike station that augments the standard finite-capacity M/M/1/K queue with an instantaneous Poisson truck-reset process; (ii) an exact greedy solver for the separable, integer-grid-discretised allocation problem programmatically $\min_\theta \sum_n F_n(\theta_n)$, justified by the strictly non-increasing marginal returns of F_n in θ_n; and (iii) a Pareto frontier of cluster failure rate against our budget, and a knee-detection result that locates our optimal operation value.

We apply this framework to a five-station Midtown Manhattan cluster fitted from December 2021 trip data, and find that an operating budget of five truck visits per hour suffices to reduce the cluster expected failure rate from 41.87 to 1.77 events per hour, a 95% reduction.

II. System Model

A. Per-Station CTMC

Consider a docking station n, with fixed dock capacity c_n. Let $s_n(t) \in \{0, 1, \ldots, c_n\}$ represent the number of bikes at the station at any given time t. The available number of open docks is then $c_n - s_n(t)$. We model the two user-driven transitions and one operator-driven transition with the truck:

- **Withdrawal:** a rider arrives wanting a bike. Modeled as a Poisson process with the rate λ_n events per hour. It is successful when $s_n > 0$ (at the transition $s_n \to s_n - 1$), and there is a stockout event when $s_n = 0$.
- **Deposit:** a rider arrives at a station to return a bike. Deposits are modeled as a Poisson process with rate μ_n. They are deemed successful when $s_n < c_n$ (transition $s_n \to s_n + 1$), and when no dock is available, a *dockblock* occurs when $s_n = c_n$.
- **Truck reset:** a rebalancing visit arrives as a Poisson process with rate θ_n in events per hour. Each visit instantaneously resets the station state to a target level t_n, where $t_n = \lfloor c_n/2 \rfloor$ unless otherwise stated.

When $\theta_n = 0$, the resulting CTMC is reduced to a finite-capacity birth-death chain with the same structure as the M/M/1/K queue with the capacity $K = c_n$. Defining the dimensionless fill intensity $\rho_n = \mu_n/\lambda_n$, the stationary distribution has the standard closed form.

$$\pi_k(n) = \frac{\rho_n^k(1-\rho_n)}{1-\rho_n^{c_n+1}}, \qquad k = 0, 1, \ldots, c_n, \tag{1}$$

for $\rho_n \neq 1$. In the balanced case where $\rho_n = 1$, the limiting distribution is uniform:

$$\pi_k = \frac{1}{c_n + 1}.$$

In real implementation, π_k is computed using a log-sum-exp normalization to avoid numerical overflow for stations with the large imbalance ratios. This is necessary in the chosen cluster because one station has $\rho \approx 14$, which can unveil the powers of ρ_n unstable.

B. Per-Station Failure Rate

The long-run rate at which the station produces failed user events is

$$F_n = \lambda_n \, \pi_0(n) + \mu_n \, \pi_{c_n}(n), \tag{2}$$

where the first term counts withdrawal attempts that arrive at an empty station (stockouts) and the second counts deposit attempts that arrive at a full station (dockblocks). F_n depends on $(\lambda_n, \mu_n, c_n, \theta_n, t_n)$ through π.

C. Truck-Reset Extension

Once θ_n is above zero, truck rebalancing brings in a new type of transition into how the station changes over time. Instead of only going state by state, stepping through withdrawals or deposits, the system may jump directly from the state k to t_n, the target level. That reset can skip several intermediate states in one transition, which means the process is arbitrarily no longer an ordinary birth-death chain. As a result, the closed-form stationary distribution in (1) can no longer apply.

For this extended process, we built a $(c_n + 1) \times (c_n + 1)$ generator matrix Q, one for each station:

$$\begin{aligned} Q_{k,k+1} &= \mu_n \quad (0 \leq k < c_n), \\ Q_{k,k-1} &= \lambda_n \quad (0 < k \leq c_n), \\ Q_{k,t_n} &\mathrel{+}= \theta_n \quad (k \neq t_n), \\ Q_{k,k} &= -\sum_{j \neq k} Q_{k,j}. \end{aligned} \tag{3}$$

Starting from the state k, a deposit moves the station to $k+1$. A withdrawal brings it down to $k-1$. Every now and then, a truck arrives and resets the station directly to t_n, as long as the station is not already at the target level. The diagonal entry is then filled in as the negative total of all other entries in that row, so that every row of Q sums to zero, as required for a valid CTMC generator matrix.

The steady-state pattern π is found by solving the equations

$$\pi Q = 0, \qquad \sum_k \pi_k = 1.$$

This is usually solved by replacing one row of $Q^\top$ with the normalization rule. This then removes the dependency issue in the stationary equations and leaves one valid probability solution. After computing π, we plug it into Equation (2) to obtain the adjusted failure rate $F_n(\theta_n)$.

This version keeps the original setup with no rebalance effort as a special case. If $\theta_n = 0$, the reset transition disappears, causing the generator matrix to become the standard finite-capacity birth-death process again. Numerical checks show that this case matches the formula in (1) to floating-point accuracy, confirming that the trucks reset model does extend the original CTMC rather than actually replacing it.

TABLE I
SELECTED FIVE-STATION MIDTOWN CLUSTER (DEC 2021, 07:00–10:00 WEEKDAYS).

Station	Type	λ (/h)	μ (/h)	ρ	c (docks)
Madison Ave & E 51 St	sink	1.25	17.90	14.36	43
W 42 St & 6 Ave	balanced	8.87	13.00	1.47	53
8 Ave & W 31 St	source	23.64	11.01	0.47	97
Broadway & W 58 St	balanced	11.65	11.14	0.96	79
W 55 St & 6 Ave	sink	5.45	13.38	2.45	77

III. DATA AND CLUSTER SELECTION

A. Trip and Station Data

We use the public Citi Bike trip archive [1] for December 2021, a representative pre-holiday month with a full set of weekday commuter mornings. Each row records the start station, end station, start and end timestamps, along other non-relevant data for our work. The data was loaded with pandas, with timestamps cast and station identifiers reconciled against the General Bikeshare Feed Specification (GBFS) station feed [3] (which provides per-station capacities c_n).

B. Cluster Selection

To show the full source–sink interaction that rebalancing acts on, a greedy clustering algorithm was run on across all stations to determine 5 stations that had the highest imbalance [4].

IV. OPTIMIZATION METHOD

A. From Capacity Reallocation to Rebalancing-Rate Allocation

The original problem we proposed was a fixed-capacity reallocation question: with $\sum c_n$ held constant, search for the integer allocation $(c_1^*, \ldots, c_N^*)$ that minimizes the cluster failure rate. For our cluster this lever had zero impact, presumably because our research data is from a system that has already optimized it. Adding or removing one dock changed F_n by less than 10^{-10}. We subsequently switched the lever to the rebalancing rate θ_n, which directly attacks the boundary probabilities through the truck-reset transition in (3), a feature that cannot be captured in our dataset.

B. Problem Statement

Given a total rebalancing budget Θ_{total} (in truck-visits per hour aggregated across the cluster), allocate it across the $N = 5$ stations to minimise long-run cluster failure:

$$\begin{aligned} \min_{\theta_1,\ldots,\theta_N \geq 0} \quad & F(\theta_1,\ldots,\theta_N) = \sum_{n=1}^{N} F_n(\theta_n) \\ \text{subject to} \quad & \sum_{n=1}^{N} \theta_n = \Theta_{\text{total}}. \end{aligned} \tag{4}$$

The objective is separable in θ_n, and each F_n is monotone non-increasing and convex in θ_n (verified numerically across the relevant range; see Section V).

C. Greedy Marginal Analysis

We worked to extend this project past the stochastic course material, utilizing a programmatic greedy marginal analysis computation. Converting the budget into discrete steps of $\delta = 0.01$, we have $\theta_n^{(0)} = 0$ for all n. At iteration k, we allocate one increment δ to the station with the largest marginal failure reduction:

$$n^{(k)} = \arg\max_n \left[F_n(\theta_n^{(k-1)}) - F_n(\theta_n^{(k-1)} + \delta) \right], \tag{5}$$

update $\theta_{n^{(k)}}^{(k)} \leftarrow \theta_{n^{(k)}}^{(k-1)} + \delta$, and continue until the cumulative budget reaches Θ_{total}. This is a well known greedy algorithm in computer science, and is proven to be optimal for the problem. We verify the non-increasing returns at every step of every run (zero violations across all reported experiments).

D. Pareto Frontier Using Nested Prefix

We can execute the greedy once at $\max(\Theta_{\text{total}})$, save the per-step trajectory, and sample it at any required budget. This turns out to be a natural consequence of the greedy algorithm [2]. This yields the full Pareto frontier $F^*(\Theta_{\text{total}})$ in a single greedy pass.

V. Results

A. Base Model

For the initial test, every station runs with its estimated rates and capacity limits, with no trucks moving between spots. Table II shows the individual failure rates for each location, split into empty-station cases and full-dock cases. Added together, the overall failure rate is

$$F_{\text{base}} = \sum_n F_n(0) = 41.87 \text{ events/hour.}$$

This totals to around 126 times that users run into issues during a three-hour weekday morning window.

The base model shows where the main problems begin. At 8 Ave & W 31 St, the cluster's strongest pickup point, stockouts happen most often. Meanwhile, dockblocks rapidly occur at drop-off heavy spots, mainly Madison Ave & E 51 St and W 55 St & 6 Ave. These trouble zones line up closely with how unbalanced each station is. Stations sending out more bikes tend to stockout, while stations receiving more bikes tend to overflow.

TABLE II
BASE-MODEL PER-STATION RESULTS AT $\theta = 0$.

Station	π_0	π_c	F_n (/h)	stockout (/h)	dockblock (/h)
Madison & E 51 St	$\sim 10^{-50}$	0.930	16.65	0.00	16.65
W 42 St & 6 Ave	$\sim 10^{-9}$	0.318	4.13	0.00	4.13
8 Ave & W 31 St	0.534	$\sim 10^{-32}$	12.62	12.62	0.00
Broadway & W 58 St	0.045	0.001	0.54	0.52	0.01
W 55 St & 6 Ave	$\sim 10^{-31}$	0.593	7.93	0.00	7.93
Total			**41.87**	**13.15**	**28.72**

TABLE III
OPTIMAL ALLOCATION AT $\Theta_{\text{TOTAL}} = 5$ EVENTS/H.

Station	θ_n^* (/h)	F_n^{base} (/h)	F_n^{opt} (/h)	Δ
Madison & E 51 St	2.42	16.65	0.87	-94.8%
W 42 St & 6 Ave	0.59	4.13	0.26	-93.8%
8 Ave & W 31 St	1.09	12.62	0.32	-97.5%
Broadway & W 58 St	0.11	0.54	0.07	-87.1%
W 55 St & 6 Ave	0.79	7.93	0.25	-96.8%
Cluster	**5.00**	**41.87**	**1.77**	$\mathbf{-95.8\%}$

B. Per-Station Response Curves

We check each station's reaction to rebalancing by testing different values of θ_n, stepping through rates from 0.1 to 10 events per hour, and then calculating $F_n(\theta_n)$ each time. Figure 1 shows that all five curves move downward steadily, so more rebalancing does not raise predicted failures. The improvements also get smaller as θ_n grows; the biggest drops happen when rebalancing is first introduced.

The stations that react most strongly are the least balanced ones. Madison Ave & E 51 St is a strong sink, with $\rho = 14.4$. 8 Ave & W 31 St is a strong source, with $\rho = 0.47$. These two stations contain the largest share of preventable failures. The boundary-probability curves show the mechanism: rebalancing cuts π_{c_n} quickly for sink stations and reduces π_0 for source stations. When a station already sits closer to balanced, its starting failure risk is smaller, so the gains from additional rebalancing do not go as far.

C. Single-Budget Optimization

Next, we evaluate a representative hourly budget of $\Theta_{\text{total}} = 5$ events per hour across the five-station cluster. When choices are made by chasing the largest immediate failure reduction under this limit, the results are shown in Table III.

When the total budget reaches 5 events per hour, failures drop sharply from 41.87 to 1.77 events per hour, a reduction of about 96 percent. The largest share of resources goes to Madison Ave & E 51 St, which causes the largest sink-side failure burden. The next largest share goes to 8 Ave & W 31 St, which creates the largest stockout burden. This shows that rebalancing targets the places where pressure builds fastest at the station boundaries.

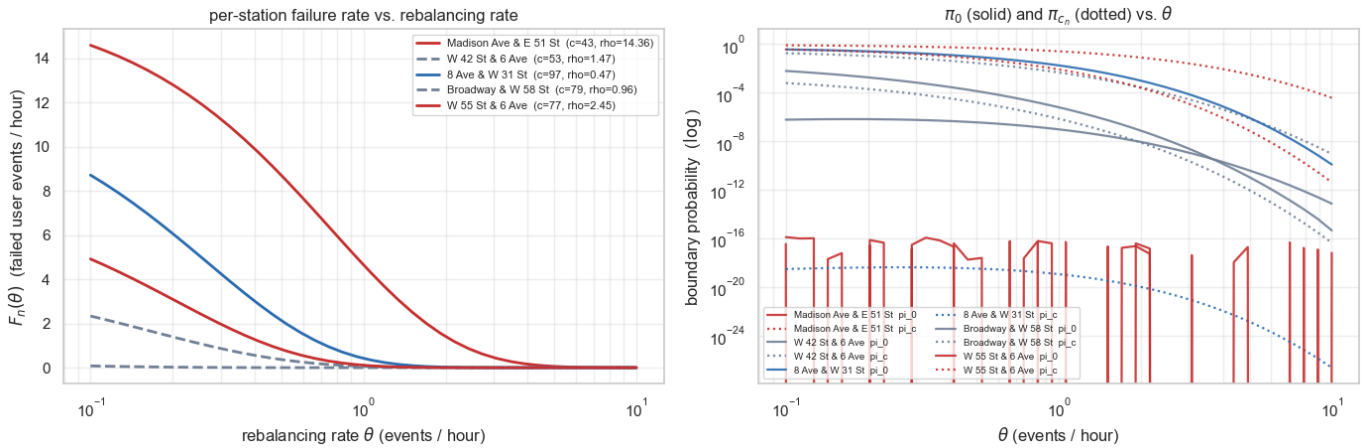

Fig. 1. Per-station response to rebalancing rate θ. **Left:** long-run failure rate $F_n(\theta)$ for each cluster station, log-x. The two extremes (Madison & E 51, 8 Ave & W 31) carry the largest reducible failure mass; the near-balanced station (Broadway & W 58) starts close to zero. **Right:** stationary boundary probabilities π_0 (solid) and π_{c_n} (dotted), log-y, showing the mechanism by which θ reduces each station's dominant failure mode.

D. Pareto Frontier and Knee

Sweeping $\Theta_{\text{total}} \in [0, 50]$ via the nested-prefix construction yields the Pareto frontier in Figure 2, left panel. The frontier is steep up to $\Theta \approx 6$ and then flattens to near zero, confirming that our chosen operating budget of $\Theta = 5$ sits on the steep, high-marginal-return portion of the curve. Beyond approximately five to six truck-visits per hour, additional rebalancing buys negligible improvement; below that range, marginal returns are large and any reduction in the budget is costly. The right panel of Figure 2 shows that as the budget grows, allocation goes first to the heaviest sink, then to the heaviest source, with the near-balanced stations always trailing.

VI. Validation and Limitations

A. Numerical Validation of the Model and Solver

To check the model and solver, two numerical tests are run. When $\theta_n = 0$, truck resets disappear, and the expanded CTMC must become identical to the standard M/M/1/K setup. That behavior is tested by matching the steady-state result π from solving $\pi Q = 0$ against the closed-form formula in (1). In every case tested, values align within 10^{-10}, showing that the updated version properly builds on the original model.

Next, we check the small drops used by the greedy solver. For the greedy allocation to hold, each added rebalancing step should bring no more gain than the one before it:

$$F_n(\theta_n) - F_n(\theta_n + \delta).$$

As θ_n increases, these marginal values should stay flat or decrease. Every test including the single-budget run, Pareto scan, rate perturbations, and target-rule adjustments showed this trend held. No violations appeared, which supports assigning resources step by step when the budget is fixed on a discrete grid.

TABLE IV
Exponential goodness-of-fit on same-day inter-arrivals. Pass requires both chi-squared $p > 0.05$ and KS $p > 0.05$.

Station	Event	n	χ^2 p	KS p	Pass
Madison & E 51	withdraw	64	0.185	0.462	✓
Madison & E 51	deposit	1212	0.000	0.000	—
W 42 & 6 Ave	withdraw	590	0.000	0.000	—
W 42 & 6 Ave	deposit	875	0.000	0.000	—
8 Ave & W 31	withdraw	1608	0.000	0.000	—
8 Ave & W 31	deposit	737	0.115	0.048	—
Broadway & W 58	withdraw	785	0.002	0.001	—
Broadway & W 58	deposit	750	0.007	0.000	—
W 55 & 6 Ave	withdraw	354	0.022	0.039	—
W 55 & 6 Ave	deposit	900	0.025	0.000	—

B. Inter-Arrival Goodness of Fit

The model treats withdrawals and deposits at each station as Poisson processes, meaning the waiting times between events should follow an exponential pattern within the morning window. To check this timing assumption, we use same-day inter-arrival samples and skip gaps that cross into the next morning, since the process resets overnight. Each station-event stream is tested using chi-squared and Kolmogorov–Smirnov goodness-of-fit checks. The results are shown in Table IV.

Nine of the ten streams reject the exponential assumption at the joint $p > 0.05$ threshold which is very consistent with the non-stationarity inside the three-hour morning window where demand rises near the beginning of the commute peak, concentrate around the middle of the window, and taper later in the morning. A single constant-rate Poisson process cannot perfectly capture that within-window pattern.

We then interpret the CTMC as a tractable average-rate model rather than a perfect description of every arrival process. The optimization results should be read as the optimal state under the stationary CTMC assumptions, not as a direct ground-truth prediction of the full Citi Bike system. Figure 3

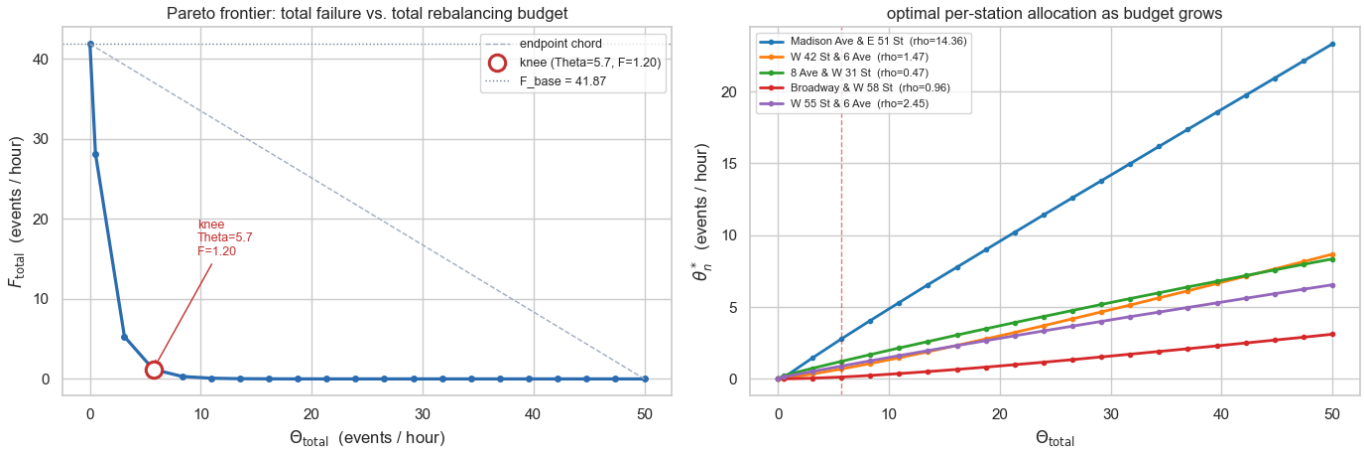

Fig. 2. **Left:** Pareto frontier of cluster failure rate vs. total rebalancing budget. The knee at $\Theta = 5.71$ (red circle) marks the transition from steep to saturated marginal returns. The endpoint chord (dashed grey) is the kneedle-method baseline. **Right:** per-station optimal allocation θ_n^* as a function of Θ_{total}. Madison & E 51 (heaviest sink) absorbs the largest share at every budget; Broadway & W 58 (near-balanced) tracks near zero.

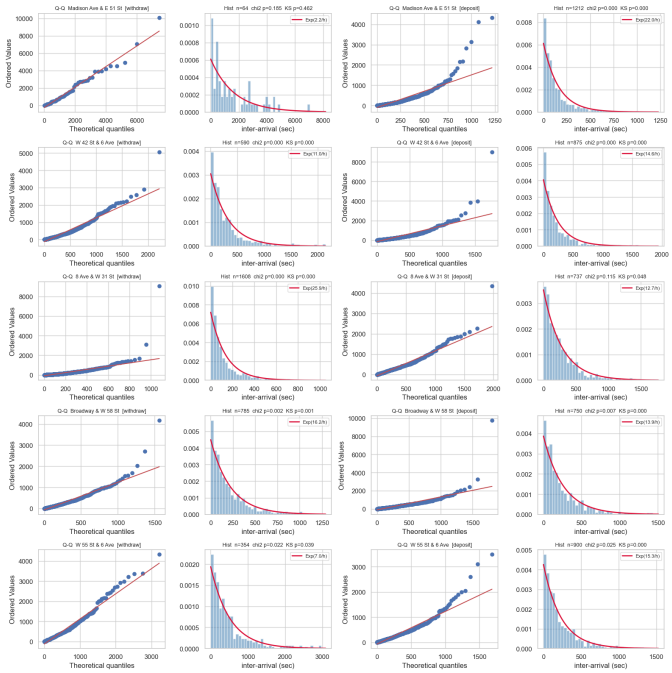

Fig. 3. Exponential goodness-of-fit diagnostics for each station-event stream. Each pair shows a Q-Q plot and a histogram with the fitted exponential density. Departures from the diagonal indicate deviations from the exponential assumption, consistent with within-window rate non-stationarity.

shows the corresponding Q-Q diagnostics and inter-arrival histograms.

C. Other Limitations

No balking model. When someone finds an empty dock or full station, they are assumed to just leave instead of going to another nearby spot. This choice keeps each station's flow separate, and mixing them by letting users switch stations is not covered here though it might come later.

Demand-side optimization only. Instead of planning truck routes, we adjust how often rebalancing visits happen. Whether a fleet can actually handle specific rates like $(\theta_1, \ldots, \theta_N)$ depends on vehicle logistics which separate from what we tackle here. That part belongs to the specific routing challenges, not our method.

VII. Conclusion and Future Work

We modeled a five-station Citi Bike cluster as independent finite-capacity Markov chains with Poisson truck-reset transitions and showed that an exact greedy marginal-analysis solver reduces cluster-wide morning-rush user failures by 95.8% at an operating budget of five truck-visits per hour, with the Pareto knee at $\Theta = 5.71$ giving a near-identical 97.1% reduction. The optimal allocation is concentrated on the cluster's two extremes: the heaviest sink and the heaviest source absorb 70% of the budget. Sensitivity is moderate to rate-estimation error and meaningful but bounded under reasonable variations of the truck target-fill rule.

One idea for future work is a user-incentive lever in which adaptive pricing discounts deposits at empty stations and surcharges deposits at full ones. This could turn riders into a self-rebalancing force, complementing the truck-rate allocation studied here.

Acknowledgment

Thank you to the amazing SYS 3060 teaching staff for guidance on model validation and advice on where to steer work, and Citi Bike NYC for releasing trip-history data that made this work possible.

References

[1] Lyft, Inc., "Citi Bike system data," 2024. [Online]. Available: https://citibikenyc.com/system-data.
[2] Claude Opus 4.7 - Anthropic PBC Available: https://claude.ai.
[3] MobilityData, "General Bikeshare Feed Specification (GBFS)," v3.0, 2023. [Online]. Available: https://github.com/MobilityData/gbfs.
[4] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.